

ASP.NET Core CON C#

11. Sesiones

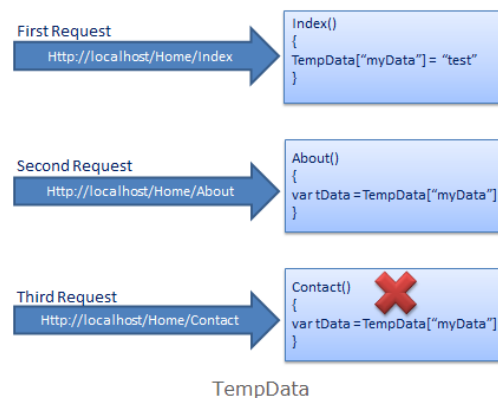
Objetos Persistentes de Servidor

La información que tenemos en los controllers/views/clases no es persistente, siempre es creada de nuevo cada vez que realizamos una petición.

Hay más colecciones, aparte de ViewBag y ViewData que nos permiten comunicar información entre el controlador/acción/vista

- **TEMPDATA:** Permite enviar información entre los controladores. Por ejemplo, podemos enviar Información entre IActionResult. Su contenido es temporal, una vez que es consumido se destruye.

Utilizaremos este diccionario para pasar datos desde una acción de un controlador a otra acción de este o de otro controlador. Los datos sólo se conservan entre una solicitud y la siguiente.



```
.  
public ActionResult Index()  
{  
    TempData["name"] = "Test data";  
    TempData["age"] = 30;  
  
    return View();  
}  
  
public ActionResult About()  
{  
    string userName;  
    int userAge;  
  
    if(TempData.ContainsKey("name"))  
        userName = TempData["name"].ToString();  
  
    if(TempData.ContainsKey("age"))  
        userAge = int.Parse(TempData["age"].ToString());  
  
    // do something with userName or userAge here  
  
    return View();  
}  
}
```

- **CACHE:** La información se almacena en el CLIENTE, es decir, el explorador Web. Se necesita un Nuget para poder almacenar dicha información. Se almacena mediante una clase llamada **IMemoryCache**. Cache va establecido por un límite de tiempo
- **COOKIES:** Las Cookies se almacenan en el explorador del Cliente. Pueden ser útiles para personalizar la Experiencia del cliente en nuestra página, nunca para almacenar información comprometida.
- **SESSION:** La información se almacena en el Servidor (Mayor seguridad). Cada usuario tendrá un Espacio de memoria distinto. Debemos tener cuidado con lo que almacenamos para no saturar el server. Session va establecida por un límite de tiempo

Sesiones

El estado de sesión ASP.NET permite almacenar y recuperar valores para un usuario a medida que el usuario navega por páginas ASP.NET en una aplicación web. HTTP es un protocolo sin estado. Esto significa que un servidor web trata cada solicitud HTTP de una página como una solicitud independiente. El servidor no retiene los valores que se utilizaron durante las solicitudes anteriores. El estado de la sesión ASP.NET identifica las solicitudes del mismo navegador durante una ventana de tiempo limitado como una sesión, y proporciona una forma de persistir los valores variables durante la duración de esa sesión. De forma predeterminada, el estado de sesión ASP.NET está habilitado para todas las aplicaciones ASP.NET.

Las variables de sesión se almacenan en un objeto **SessionStateItemCollection** que se expone a través de la propiedad **Session** del objeto HttpContext en los controladores. En las vistas, las variables de sesión actuales se exponen a través de la propiedad Session del objeto Context.

Las sesiones se identifican mediante un identificador único que se puede leer utilizando la propiedad SessionID. Cuando el estado de sesión está habilitado para una aplicación ASP.NET, cada solicitud de una página en la aplicación se examina en busca de un valor de SessionID enviado desde el navegador. Si no se proporciona ningún valor de SessionID, ASP.NET inicia una nueva sesión y el valor de SessionID para esa sesión se envía al navegador con la respuesta.

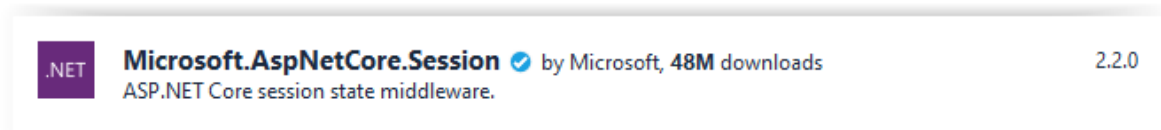
Por defecto, los valores de SessionID se almacenan en una cookie. Sin embargo, también puede configurar la aplicación para almacenar valores de SessionID en la URL para una sesión "sin cookies".

Una sesión se considera activa siempre que las solicitudes continúen realizándose con el mismo valor de SessionID. Si el tiempo entre solicitudes para una sesión en particular excede el valor de tiempo de espera especificado en minutos, la sesión se considera expirada. Las solicitudes realizadas con un valor de SessionID caducado dan como resultado una nueva sesión.

En principio, las sesiones sólo nos permiten almacenar datos en binario

```
HttpContext.Session.Set("KEY", value);  
var dato = HttpContext.Session.Get("KEY");
```

Para poder trabajar con datos primitivos además de binarios, debemos instalar el Nuget



Más adelante veremos como hacer para poder almacenar objetos, listas...

Para poder usar las sesiones en nuestro proyecto, daremos los siguientes pasos:

1. Instalar el Nuget anterior
2. Resolvemos las dependencias

```
// Add services to the container.  
//NECESITAMOS EL SERVICIO DE MEMORIA DE CACHE  
builder.Services.AddDistributedMemoryCache();  
//UNA SESSION FUNCIONA POR TIEMPO DE INACTIVIDAD  
builder.Services.AddSession(options =>  
{  
    options.IdleTimeout=TimeSpan.FromMinutes(10);  
});
```

```
App.UseSession();
```

Es imprescindible que esta última instrucción se ponga antes del MapRoute

Ejemplo: Véase EjemploSesion_1

Veamos como almacena tipos de datos no primitivos (objetos)

Debemos instalar otro NuGet



Lo utilizaremos para crear una clase que **extienda** de Session que implemente los métodos GetObject y SetObjet para que la sesión gestione tipos de datos Object

Pasos que daremos:

1. Crear una carpeta llamada Extensions
2. Crear una clase estática con los dos métodos mencionados arriba también estáticos
3. El método SetObject recibe como parámetros la clase que vamos a extender, el nombre y valor del objeto que almacenaremos en la sesión. Transforma el objeto en un string json
4. El método GetObject que retornará un Objeto de tipo T
5. Debemos añadir las mismas dependencias que en el caso de tipos primitivos

Es importante tener en cuenta que para usar estos métodos en las vistas hay que importar el namespace de la clase extensión que hemos creado en el archivo `_ViewsImports`

Ejemplo: Véase EjemploSesión_2

De forma análoga podemos almacenar en una sesión colecciones (arrays, listas....)

Para eliminar las variables de una sesión usaremos el método Clear